

Hardware Accelerated Machine Learning



WARUM HAML?

Übliche Prozessoren:

- konventionelle Prozessoren nur bedingt für ML optimiert
- Effizienzverluste bei stark parallelen Operationen

FPGA:

- rekonfigurierbare Hardware durch FPGAs
- flexible Anpassung an die jeweilige Anwendung
- Potenzial für höhere Effizienz und Leistung

VHDL:

- Entwicklung spezialisierter Hardware
- Synthese für FPGA-Hardware



WAS HABEN WIR GEMACHT?

01 k-Nearest-Neighbors

- Implementierung eines KNN-Klassifikators in VHDL
- Untersuchung der FPGA-Umsetzung des Verfahrens

03 Sudoku-Solver

- Implementierung eines Sudoku-Solvers
- Verwendung eines genetischen Algorithmus

02 Lineare Regression

- Implementierung einer linearen Regression in Hardware
- Verwendung eines genetischen Algorithmus

04 Vergleichbarer Python Code

- Entwicklung entsprechender Python-Implementierungen
- Vergleich Software- und Hardwareimplementierung

VORAUSSETZUNGEN

Parallelität

- parallele Verarbeitung mehrerer Datenpunkte
- SIMD: gleiche Operation auf mehreren Datenwerten
- Parallelisierung auch innerhalb eines Datenpunkts

Pipelining

- Aufteilung der Berechnungen in mehrere Stufen
- gleichzeitige Verarbeitung mehrerer Datenpunkte
- unterschiedliche Pipeline-Phasen parallel aktiv

Arithmetik

- Verwendung von Fixed-Point-Zahlen
- Beschränkung auf Addition, Subtraktion und Multiplikation
- Verzicht auf Floating-Point, Division, Wurzeln und Potenzen

VORAUSSETZUNGEN

Streaming

- direkte Weitergabe durch die Pipeline
- Minimierung unnötiger Speicherzugriffe
- hoher Durchsatz durch durchgängigen Datenfluss

Agentenbasis*

- Aufteilung großer Probleme auf mehrere Agenten/FPGAs
- jeder Agent bearbeitet ein Teilproblem (z. B. Teilpopulation)
- z. B. Einsatz in genetischen Algorithmen
- möglichst geringe Kommunikation zwischen Agenten

K-NEAREST-NEIGHBORS

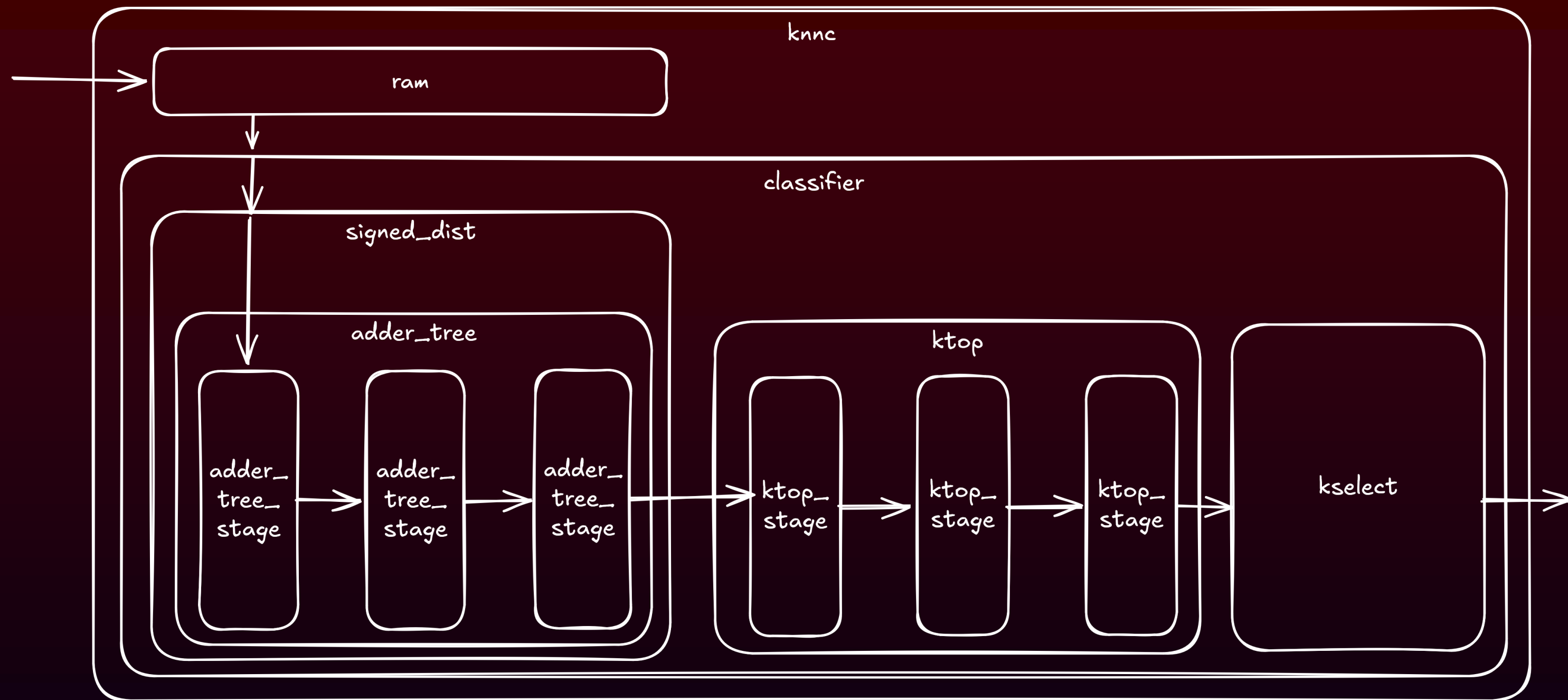
Umsetzbarkeit



- guter Kandidat für Umsetzung mit VHDL
- größtenteils voneinander unabhängige Berechnungen für verschiedene Datenpunkte
 - gut parallelisierbar
 - Pipelining und Streaming
 - beste Kandidaten während Berechnung ermitteln
- passende Arithmetik
 - Vergleich von Distanzen, Fixed-Point
 - Distanzberechnung (euklidisch)

K-NEAREST-NEIGHBORS

Struktur



K-NEAREST-NEIGHBORS

Ergebnisse

(Beispiel zur Klassifizierung der Bananen-Qualität)

Performance

- **Software** (i5-1335U; bis zu 4,6 GHz; sklearn): 160 ms
- **Hardware** (Virtex-6; 357 MHz): 34 ms

→ Beschleunigung um ca. Faktor 5
(trotz 14 Jahre älterer Hardware)

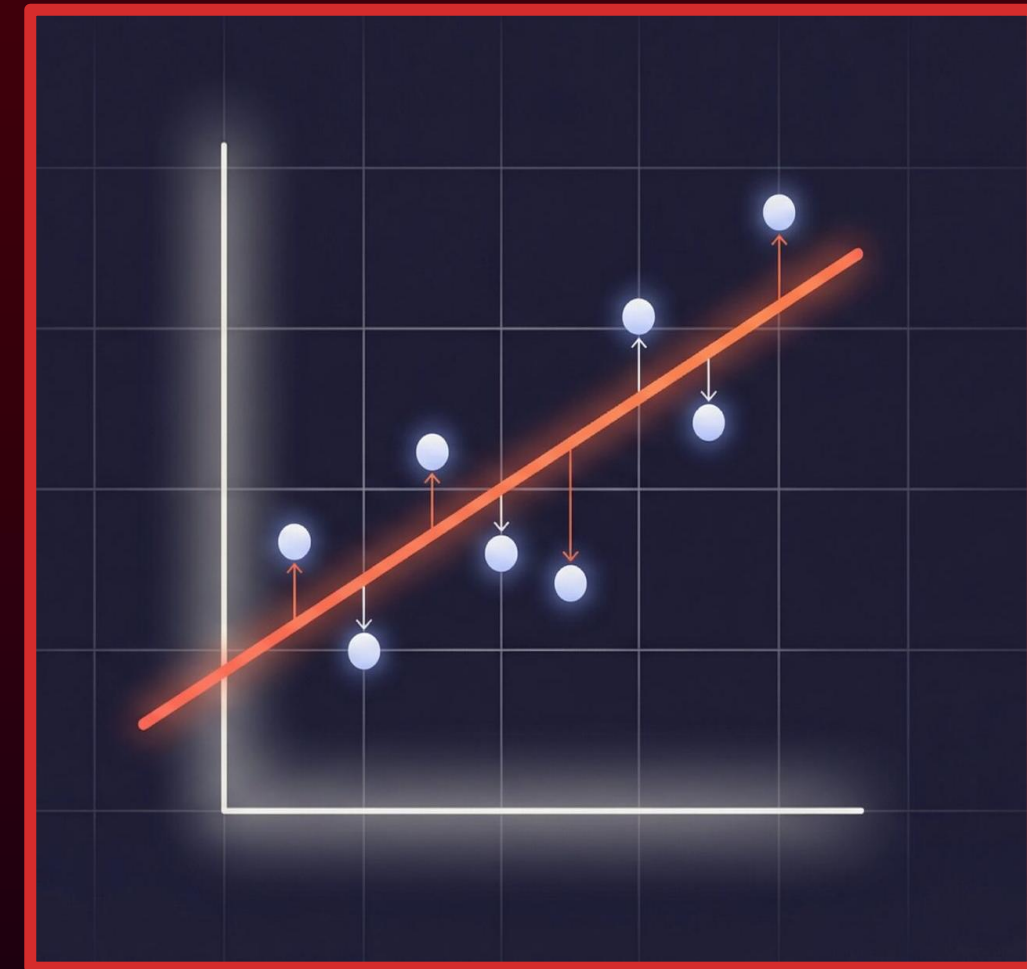
Qualität

- **Software**
 - Accuracy 97,6 %
 - Precision 97,9 %
 - Recall 97,3 %
- **Hardware**
 - Accuracy 97,4 %
 - Precision 98,3 %
 - Recall 96,4 %

LINEARE REGRESSION

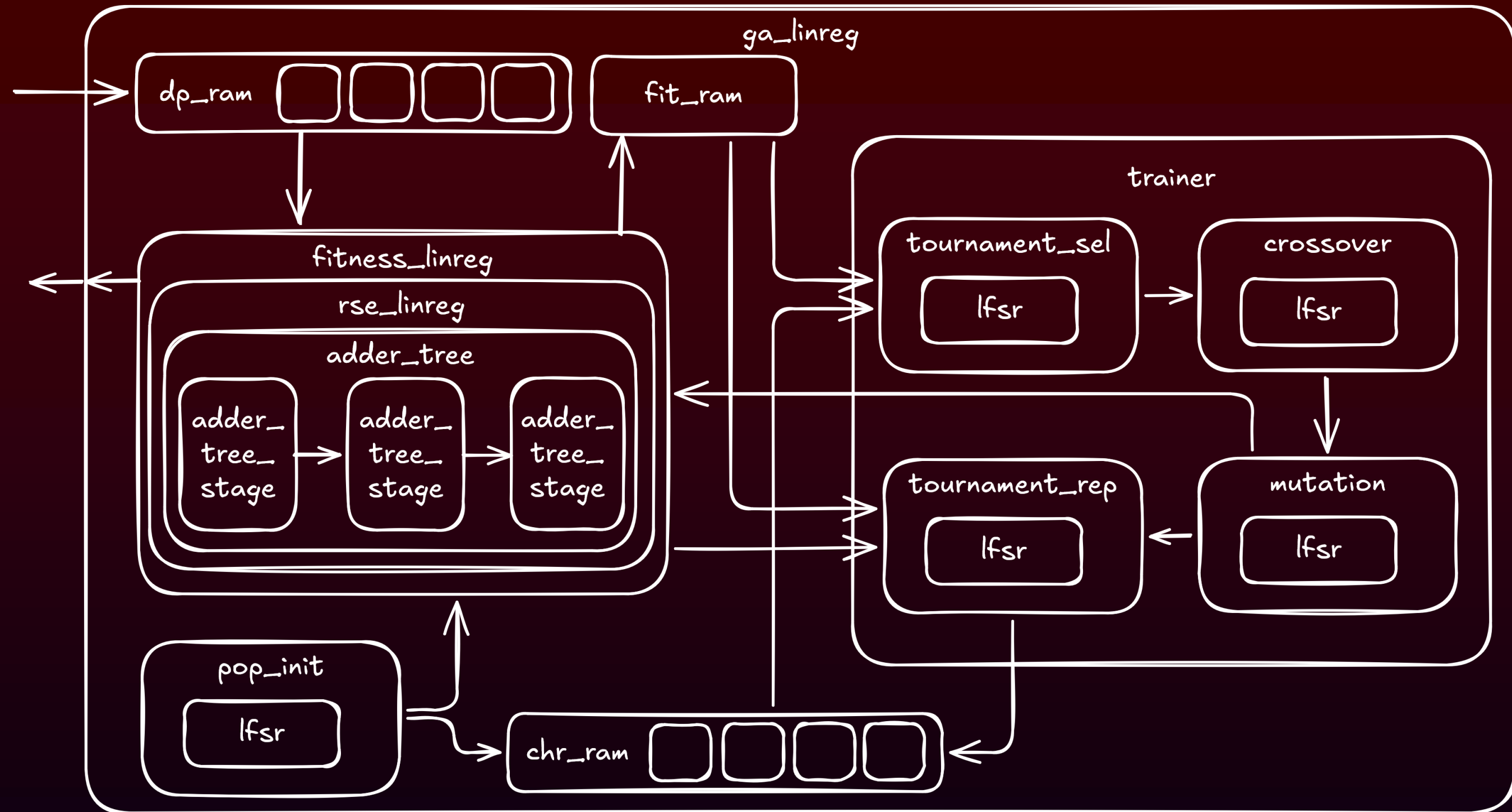
Umsetzbarkeit

- mit Einschränkungen umsetzbar
- sehr gut parallelisierbar*
 - parallele Anpassung der Chromosome
 - parallele Berechnung der Fitness
 - Einschränkung: RAM
- Pipelining + Streaming für Fitness-Funktion geeignet
- Probleme: Zufall, Speicherbedarf, Auswahl der besten Individuen, aufwendige Fitness-Funktion, Wertebereich
- agentenbasiert umsetzbar*
 - Teil-Populationen, Planetensystem



LINEARE REGRESSION

Struktur



LINEARE REGRESSION

Ergebnisse

(Beispiel-Datensätze *linear_regression{1,2}.csv*)

Performance

- **Software** (Apple M3; 4,05 GHz; PyGAD)
 - *linear_regression1*: 280 ms
 - *linear_regression2*: 277 ms
- **Hardware** (Virtex-6; 364 MHz)
 - *linear_regression1*: 0,55 ms
 - *linear_regression2*: 0,53 ms

→ Beschleunigung um mehr als Faktor 500
(trotz 14 Jahre älterer Hardware)

Qualität

- **Software**
 - *linear_regression1*: $y = -2,60 * x + 53,30$
 - *linear_regression2*: $y = 36,25 * x - 5,16$
- **Hardware**
 - *linear_regression1*: $y = -2,60 * x + 74,98$
 - *linear_regression2*: $y = 36,25 * x - 7,01$

SUDOKU SOLVER

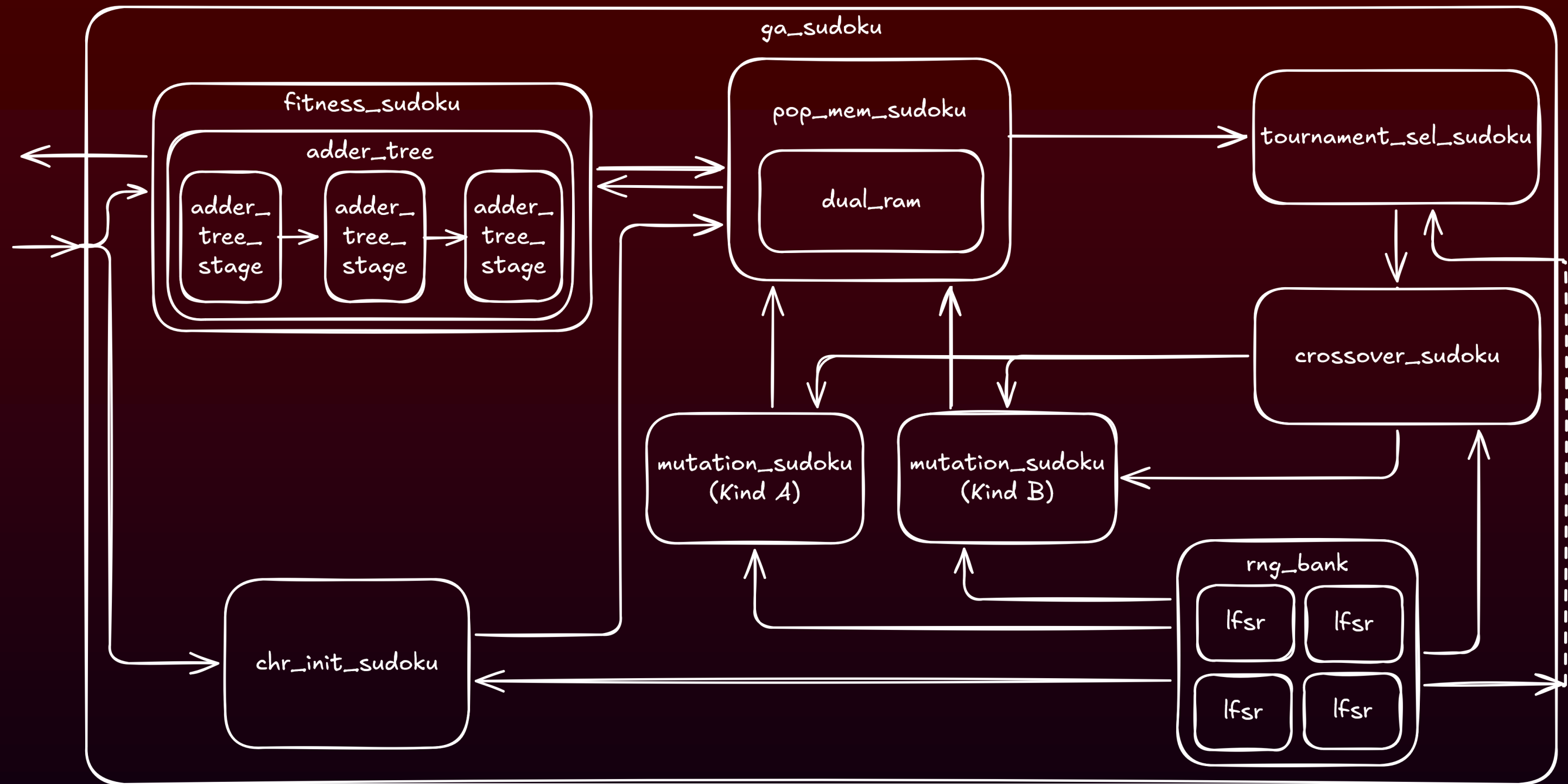
Umsetzbarkeit

- generell gelten die Überlegungen und Einschränkungen der Linearen Regression
- problemspezifische Optimierungen
 - parallele Berechnung der Fitness selbst
 - angepasste Crossover-/Mutationsstrategie (blockweise)

7			5	8	2	9	3	4
2		5		1	9			7
		3				2		1
	3	7	1		6	4	2	5
4	9		7				1	
	5	2		3	8	7		
	2			5	7	1	9	6
5	7	9	2	6				3
6				4	3	5	7	2

SUDOKU SOLVER

Struktur



SUDOKU SOLVER

Ergebnisse

Performance

- **Software** (i5-1335U; bis zu 4,6 GHz; chinyan)
- **Hardware** (Kintex-7, 68 MHz)

→ Beschleunigung um mehr als Faktor 4200
(trotz 12 Jahre älterer Hardware)

Leistungsvergleich

Sudoku-Easy `ga_sudoku_easy_tb`

Implementierung	Populationsgröße	Generationen bis zur Lösung	Benötigte Zeit
Software (Python)	128	12	3.080,00 ms
Hardware (VHDL)	128	10	0,73 ms

Sudoku-Medium `ga_sudoku_mid_tb`

Implementierung	Populationsgröße	Generationen bis zur Lösung	Benötigte Zeit
Software (Python)	1024	188	49.340,00 ms
Hardware (VHDL)	1024	nach 27 stagniert; best_fit=3	16 ms

Sudoku-Hard `ga_sudoku_hard_tb`

Implementierung	Populationsgröße	Generationen bis zur Lösung	Benötigte Zeit
Software (Python)	2048	138	534.080,00 ms
Hardware (VHDL)	2048	nach 47 stagniert; best_fit=2	43 ms

ERGEBNISSE

LOHNT SICH HARDWARE ACCELERATION?

- wesentlich höherer Implementierungsaufwand
- Geschwindigkeitsvorteil sehr unterschiedlich
- deterministische Laufzeit (echtzeitfähig)
- lohnenswert, aber nicht uneingeschränkt

EXISTIEREN SINNVOLLE ANWENDUNGSFÄLLE?

- Inferenz (Vorhersage)
- Regressionen (Linear, Logistisch, Polynomiell*)
- k-Nearest-Neighbors
- genetische Algorithmen
- Neuronale Netze
- Swarm Optimization
- Simulated Annealing

ERGEBNISSE

WELCHE KRITERIEN SOLLTEN BEACHTET WERDEN?

- Eignung für Pipelining, Streaming und Parallelisierung
- Arithmetik (Genauigkeit, Wertebereiche, Operationen)
- Datenmenge/RAM und Komplexität der Aufgabe prüfen
- Vermeidung von Verzweigungen
- Entwicklungsaufwand bewerten

SIND AGENTEN- ARCHITEKTUREN SINNFÜHREND?

- Teil-Problem muss groß genug sein
- Verteilung von Teil-Problemen an Agenten (FPGAs)
- z. B. Teil-Populationen, Cluster, Planetensysteme

REFLEXION

WAS HABEN WIR IN UNIT 2 GELERNT?

- VHDL und FPGA-basierte Hardwareentwicklung
- Grundlagen von Hardware-Design für Machine Learning (Parallelisierung, Pipelining, Streaming, Arithmetik)
- Durchsatz- vs. Latenzoptimierung (ML eher durchsatz-, ALU eher latenzoptimiert)
- verschiedene Architekturansätze (FSM/Automat, Pipeline)

WANN WAR GROßER FORTSCHRITT ZU VERMERKEN?

- **Anfang:** erste VHDL-Implementierungen (z. B. kNN) zur Umsetzung und Verifikation der Konzepte
 - **Zwischenphase:** Fokus auf andere Projekte und kleinere Optimierungen Module
 - **Später:** komplexere Algorithmen (z. B. Lineare Regression, Sudoku)
-

REFLEXION

WAS NEHMEN WIR MIT?

- Abweichung bei Hardwareimplementierungen im Vergleich zu Software
- Abwägung zwischen Komplexität, Performance und Funktionsumfang (z. B. Hardware vs. Software, Fixed-Point vs. Floating-Point, FSM vs. Pipeline)
- Erkenntnis, dass sich rechenintensive Probleme oft sehr effizient in Hardware umsetzen lassen
- Motivation, weitere Probleme/Algorithmen auf Hardware-Basis zu implementieren

VIELEN DANK!



<https://github.com/IP-TeamC/haml>



<https://ip-teamc.github.io/haml/>

